

# ANNS 项目实验报告

## 基于 TunableHNSW 的高性能向量检索研究

2025 秋季数据结构 (H)

夏澄奕

### 目录

|          |                      |          |
|----------|----------------------|----------|
| <b>1</b> | <b>项目简述</b>          | <b>2</b> |
| 1.1      | 任务定义                 | 2        |
| 1.2      | 系统框架                 | 2        |
| <b>2</b> | <b>HNSW 算法深度解析</b>   | <b>2</b> |
| 2.1      | 算法简介                 | 2        |
| 2.2      | 复杂度与性能分析             | 3        |
| 2.3      | 底层实现优化               | 3        |
| 2.3.1    | SIMD 指令集加速           | 3        |
| 2.3.2    | 图存储扁平化               | 3        |
| 2.3.3    | 内存预取                 | 3        |
| 2.3.4    | 多线程并行构建              | 4        |
| 2.3.5    | 启发式剪枝策略              | 4        |
| 2.4      | 分析指标                 | 4        |
| <b>3</b> | <b>优化策略研究</b>        | <b>4</b> |
| 3.1      | 有效优化尝试               | 4        |
| 3.1.1    | 适应性 Beam Search      | 4        |
| 3.1.2    | 基于 std::pmr 的内存局部性优化 | 4        |
| 3.2      | 无效或负面优化分析            | 5        |
| 3.2.1    | 向量量化                 | 5        |
| 3.2.2    | 倒排文件索引               | 5        |
| 3.2.3    | 高层 Beam Search       | 5        |
| <b>4</b> | <b>实验结果</b>          | <b>6</b> |

# 1 项目简述

## 1.1 任务定义

在高维空间数据检索任务中,最近邻搜索即 NNS 的核心目标是从包含  $N$  个  $D$  维向量的数据集  $\mathcal{S}$  中,针对给定的查询向量  $q$ ,检索出与  $q$  距离最近的向量  $x^*$ 。其中数据集  $\mathcal{S}$  为  $\{x_1, x_2, \dots, x_N\} \subset \mathbb{R}^D$ , 查询向量  $q \in \mathbb{R}^D$ 。该数学定义如下所示:

$$x^* = \arg \min_{x \in \mathcal{S}} \text{dist}(q, x) \quad (1)$$

在本实验中,我们统一采用欧几里得距离,也就是常说的 L2 距离作为度量标准。

鉴于在大规模高维数据集中进行精确搜索的计算成本极高,近似最近邻搜索即 ANNS 应运而生。ANNS 旨在召回率 Recall 与检索延迟之间寻找平衡,在允许微小精度损失的前提下,构建高效的索引结构。我们的目标是实现远低于  $O(N)$  的搜索时间复杂度,同时确保结果的召回率维持在较高水平。

## 1.2 系统框架

为了全面评估不同算法与优化策略的有效性,本项目开发了一个模块化且可扩展的 ANNS 测试框架,命名为 **TunableHNSW**。该框架允许用户灵活配置各类优化开关与参数,支持从底层数据结构到上层检索逻辑的全方位评测。项目的整体目录结构组织如下:

- **data\_o/**: 数据管理模块,包含 SIFT 和 GloVe 等标准数据集。其中包括作为底库向量的 base 文件、查询向量 sample 文件以及作为标准答案 Ground Truth 的 label 文件。
- **src/**: 核心源代码目录。
  - **TunableHNSW/**: 核心算法实现。
    - \* **HNSW.h** 与 **IVF.h**: 分别实现了 HNSW 图结构与倒排索引结构。
    - \* **Distance.h**: 封装了基于 SIMD 优化的距离计算函数。
    - \* **Metrics.h**: 负责召回率、延迟等关键性能指标的计算。
    - \* **PQ.h**、**OPQ.h** 与 **SQ.h**: 实现了多种向量量化编码策略。
    - \* **std::pmr** 相关优化: 集成在内存管理模块中。
  - **TestBench.cpp** 与 **Main.cpp**: 负责测试流程的调度与基准测试的执行。

# 2 HNSW 算法深度解析

## 2.1 算法简介

分层可导航小世界图,简称 HNSW,是当前最为先进的基于图的 ANNS 算法之一。它借鉴了跳表的分层思想,将图结构构建为多层级的形式:

- **底层 Layer 0:** 该层包含数据集中的所有节点。它构建了一个近似的 Delaunay 图，确保了图的连通性，主要用于执行高精度的局部细化搜索。
- **上层 Layer > 0:** 每一层都是下一层节点的随机子集，层级越高则节点分布越稀疏。这些稀疏层构成了搜索的“高速公路”，允许查询点在图中进行长距离的贪心跳转，从而快速逼近目标所在的区域。

整个搜索过程采用自上而下的策略：从最高层出发，通过贪心算法快速定位到该层距离查询点最近的节点，并将其作为进入下一层的入口点。这一过程逐层向下递进，直至降至第 0 层完成最终的精细搜索。

## 2.2 复杂度与性能分析

**理论复杂度** HNSW 的构建时间复杂度为  $O(N \log N)$ ，而搜索时间复杂度理论上为  $O(\log N)$ 。

**实际性能瓶颈** 尽管理论搜索复杂度呈对数级，但由于高层导航主要采用贪心策略且节点稀疏，实际检索耗时绝大部分集中在第 0 层。虽然第 0 层内的搜索在理论上可视作相对于总数据量  $N$  的常数级  $O(1)$ ，但其实际开销与探索因子参数  $Ef$  高度相关。

具体而言，在第 0 层的导航过程中，主要开销来源于两部分：一是维护候选节点堆，其耗时约  $O(Ef \log Ef)$ ；二是处理邻居节点的距离计算与剪枝，其耗时约  $O(M_0 Ef^{\frac{d-1}{d}})$ 。因此，第 0 层的近似时间复杂度可表示为：

$$O(Ef \log Ef + M_0 Ef^{\frac{d-1}{d}})$$

## 2.3 底层实现优化

为了进一步挖掘硬件性能，我们在标准算法基础上实施了多项底层优化。

### 2.3.1 SIMD 指令集加速

距离计算是高维向量检索中最频繁的操作，也是性能热点。我们利用 x86 架构的 AVX2 或 AVX-512 指令集，通过向量化寄存器并行处理多个维度的浮点运算，显著提升了 L2 距离计算的吞吐量。

### 2.3.2 图存储扁平化

传统的链式邻接表会导致严重的内存碎片化。我们将图的存储结构重构为扁平化数组，使邻接关系在物理内存上连续分布。这种布局极大地提升了内存访问的局部性，减少了 CPU 缓存未命中 Cache Miss 的概率。

### 2.3.3 内存预取

在遍历图结构时，我们引入了软件预取技术。在处理当前节点计算的同时，提前将待访问邻居节点的数据加载至高速缓存中，从而掩盖内存访问的高延迟。

### 2.3.4 多线程并行构建

在索引构建阶段，利用线程库实现了多线程并行构图，大幅缩短了大规模数据集的索引建立时间。

### 2.3.5 启发式剪枝策略

为了控制图的度数并保持良好的导航性能，我们在建图过程中引入了基于相对邻域图 RNG 的启发式剪枝策略。当且仅当候选邻居与当前节点的距离小于该邻居与现有连接集中任意节点的距离时，才保留该连接。数学表达如下：若存在  $Neighbor$  使得  $\|Neighbor, Node\| < \|This, Node\|$ ，则认为  $This$  节点是冗余的并予以剪除。该策略有效降低了图的复杂度，避免了“超级节点”的产生。

## 2.4 分析指标

实验主要关注以下核心指标：

- **效率指标：**包括建图时间 Build Time 与搜索延迟 Search Time。
- **精度指标：**即搜索准确率，通常用 Accuracy 或 Recall@K 表示。
- **行为分析：**各层级搜索时间的分布情况。

## 3 优化策略研究

### 3.1 有效优化尝试

#### 3.1.1 适应性 Beam Search

标准 HNSW 算法依赖固定的 `efSearch` 参数控制候选队列大小，缺乏灵活性。我们在 `TunableHNSW` 中引入了动态终止机制：在搜索过程中实时监控候选集的变化，一旦在预设步数内前 10 个最优节点不再发生更新，即判定算法已收敛并提前终止搜索。该策略在保证召回率的同时减少了冗余计算。

#### 3.1.2 基于 `std::pmr` 的内存局部性优化

针对图算法频繁随机访问内存的特性，我们利用 C++17 引入的多态内存资源即 `std::pmr` 重构了内存管理模块：

- 使用 `std::pmr::synchronized_pool_resource` 替代默认的堆分配器，减少系统调用开销。
- 将邻接表和节点数据紧凑地分配在连续的大块内存中，强制实现空间局部性。

需要特别指出的是，在最终的整合测试表中，由于 `TunableHNSW` 测试框架本身的封装开销限制了内存优化的潜力，导致数据提升不明显。但在我们单独编写的独立 Solution 测试中，该优化方案展现出了约 **30%** 的搜索速度提升。

## 3.2 无效或负面优化分析

### 3.2.1 向量量化

实验表明，在 SIFT-1M 和 GloVe 数据集上，对向量维度进行的有损压缩，如 PQ8、OPQ8 和 SQ16 等方法，导致了严重的精度下降。这一现象可以用 Johnson-Lindenstrauss 引理进行理论解释：

Given  $0 < \epsilon < 1$ , a set  $X$  of  $N$  points in  $\mathbb{R}^n$ , and an integer  $k > \frac{8(\ln N)}{\epsilon^2}$ , there is a linear map  $f: \mathbb{R}^n \rightarrow \mathbb{R}^k$  such that for all  $u, v \in X$ :

$$(1 - \epsilon)\|u - v\|^2 \leq \|f(u) - f(v)\|^2 \leq (1 + \epsilon)\|u - v\|^2$$

for all  $u, v \in X$ .

带入 SIFT-1m 和 GloVe 数据集的参数得损失率最高为 92.923% 和 105.769%。实际测试中，所有量化方法均导致了搜索时间的增加，这主要是源于额外的解码开销，同时也带来了精度的显著滑坡。

### 3.2.2 倒排文件索引

我们尝试在 HNSW 之外引入倒排文件索引结构 IVF，期望通过 K-Means 聚类缩小搜索范围。该尝试失败的主要原因在于 SIFT 和 GloVe 数据集的高维特征分布较为均匀，简单的聚类算法难以有效划分出明确的簇。为了维持可接受的召回率，系统被迫扫描大量的聚类中心，导致筛选开销抵消了 IVF 的速度优势。经过尝试，使用了软聚类的改进方法，也未能显著提升性能。

### 3.2.3 高层 Beam Search

我们尝试将 HNSW 上层导航的贪心搜索替换为带有小 `ef` 值的 Beam Search，结果并不理想。

- **边际收益递减：**HNSW 的小世界特性已保证了贪心路由的高效性。引入 Beam Search 虽能微调 Layer 0 的入口点，但维护优先队列带来的 CPU 分支预测失败和内存读写开销，远超其节省的微量距离计算时间。
- **目标导向不同：**高层导航的目的是“快速逼近”，而非“精确求解”。第 0 层才需要通过 Beam Search 寻找绝对最优的前 K 个解并向外扩散。在高层使用多入口点策略属于过度优化。

## 4 实验结果

表 1: 不同算法与优化策略的性能对比

| Algorithm     | BuildTime<br>(s) | SearchTime<br>(ms) | Accuracy<br>(%) |
|---------------|------------------|--------------------|-----------------|
| BruteForce    | 0.56 / 0.53      | 129.07 / 132.05    | 100 / 100       |
| HNSW          | 320.57 / 2766.08 | 0.45 / 2.89        | 99.30 / 99.44   |
| HNSW+         |                  |                    |                 |
| DynamicSearch | 343.11 / 2762.14 | 0.54 / 2.35        | 99.01 / 99.13   |
| HNSW+         |                  |                    |                 |
| OPQ8&rerank   | 396.31 / 2780.98 | 0.7233 / 4.11      | 98.96 / 98.49   |
| HNSW+         |                  |                    |                 |
| SQ16          | 335.21 / 2774.81 | 1.76 / 8.26        | 99.29 / 99.25   |
| HNSW+         |                  |                    |                 |
| IVF           | 8.39 / 12.08     | 1.85 / 6.37        | 85.48 / 72.71   |
| HNSW+         |                  |                    |                 |
| PMR*          | 317.93 / 2635.42 | 0.47 / 3.09        | 99.32 / 99.43   |

\* 注：表中的 PMR 数据受限于测试框架开销。独立测试显示有约 30% 的性能提升。

表 2: 底层搜索开销与距离计算次数统计

| Algorithm     | Layer0            | Average                 |
|---------------|-------------------|-------------------------|
|               | SearchTime<br>(%) | Distance<br>Calculation |
| BruteForce    | NA                | NA                      |
| HNSW          | 94.73 / 88.45     | 200112.88 / 2186990.68  |
| HNSW+         |                   |                         |
| DynamicSearch | 94.86 / 97.96     | 199805.65 / 2182272.66  |
| HNSW+         |                   |                         |
| OPQ8&rerank   | 86.76 / 95.60     | 199933.72 / 2190428.08  |
| HNSW+         |                   |                         |
| SQ16          | 95.13 / 97.67     | 199630.58 / 2184929.68  |
| HNSW+         |                   |                         |
| IVF           | 2.33 / 2.20       | 3161.53 / 4275.26       |
| HNSW+         |                   |                         |
| PMR           | 95.23 / 98.54     | 199481.95 / 2186556.45  |